Programação 2

Aula 7 - Estruturas de Dados: arrays e arraylists

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Recapitulando

	Time
(c) C	1.00
(c) Rust	1.04
(c) C++	1.56
(c) Ada	1.85
(v) Java	1.89
(c) Chapel	2.14
(c) Go	2.83
(c) Pascal	3.02
(c) Ocaml	3.09
(v) C#	3.14
(v) Lisp	3.40
(c) Haskell	3.55
(c) Swift	4.20
(c) Fortran	4.20
(v) F#	6.30
(i) JavaScript	6.52
(i) Dart	6.67
(v) Racket	11.27
(i) Hack	26.99
(i) PHP	27.64
(v) Erlang	36.71
(i) Jruby	43.44
(i) TypeScript	46.20
(i) Ruby	59.34
(i) Perl	65.79
(i) Python	71.90
(i) Lua	82.91

	Mb
(c) Pascal	1.00
(c) Go	1.05
(c) C	1.17
(c) Fortran	1.24
(c) C++	1.34
(c) Ada	1.47
(c) Rust	1.54
(v) Lisp	1.92
(c) Haskell	2.45
(i) PHP	2.57
(c) Swift	2.71
(i) Python	2.80
(c) Ocaml	2.82
(v) C#	2.85
(i) Hack	3.34
(v) Racket	3.52
(i) Ruby	3.97
(c) Chapel	4.00
(v) F#	4.25
(i) JavaScript	4.59
(i) TypeScript	4.69
(v) Java	6.01
(i) Perl	6.62
(i) Lua	6.72
(v) Erlang	7.20
(i) Dart	8.64
(i) Jruby	19.84

	Energy
(c) C	1.00
(c) Rust	1.03
(c) C++	1.34
(c) Ada	1.70
(v) Java	1.98
(c) Pascal	2.14
(c) Chapel	2.18
(v) Lisp	2.27
(c) Ocaml	2.40
(c) Fortran	2.52
(c) Swift	2.79
(c) Haskell	3.10
(v) C#	3.14
(c) Go	3.23
(i) Dart	3.83
(v) F#	4.13
(i) JavaScript	4.45
(v) Racket	7.91
(i) TypeScript	21.50
(i) Hack	24.02
(i) PHP	29.30
(v) Erlang	42.23
(i) Lua	45.98
(i) Jruby	46.54
(i) Ruby	69.91
(i) Python	75.88
(i) Perl	79.58

What Are the Greenest Programming Languages? | by Kesk -*- | CodeX | Medium

Objetos: Criação: Construtores

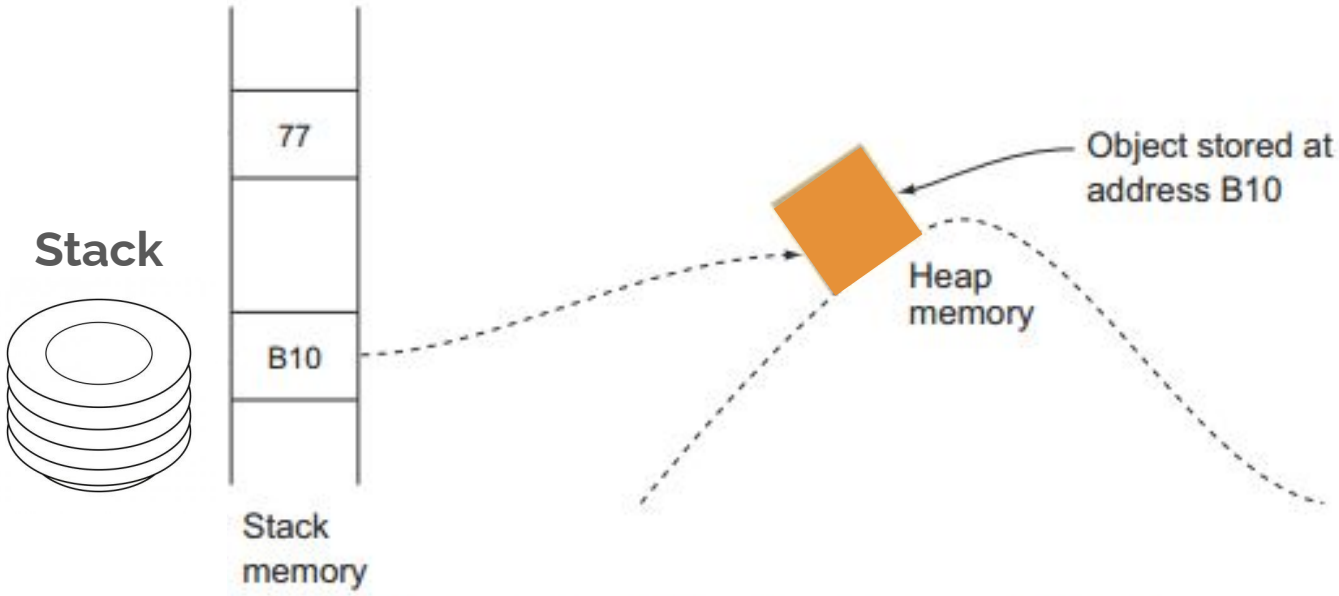
No código...

```java
public class Cor {
    int r;
    int g;
    int b;

    Cor(int r, int g, int b) {
        this.r = r;
        this.g = g;
        this.b = b;
    }
}
```

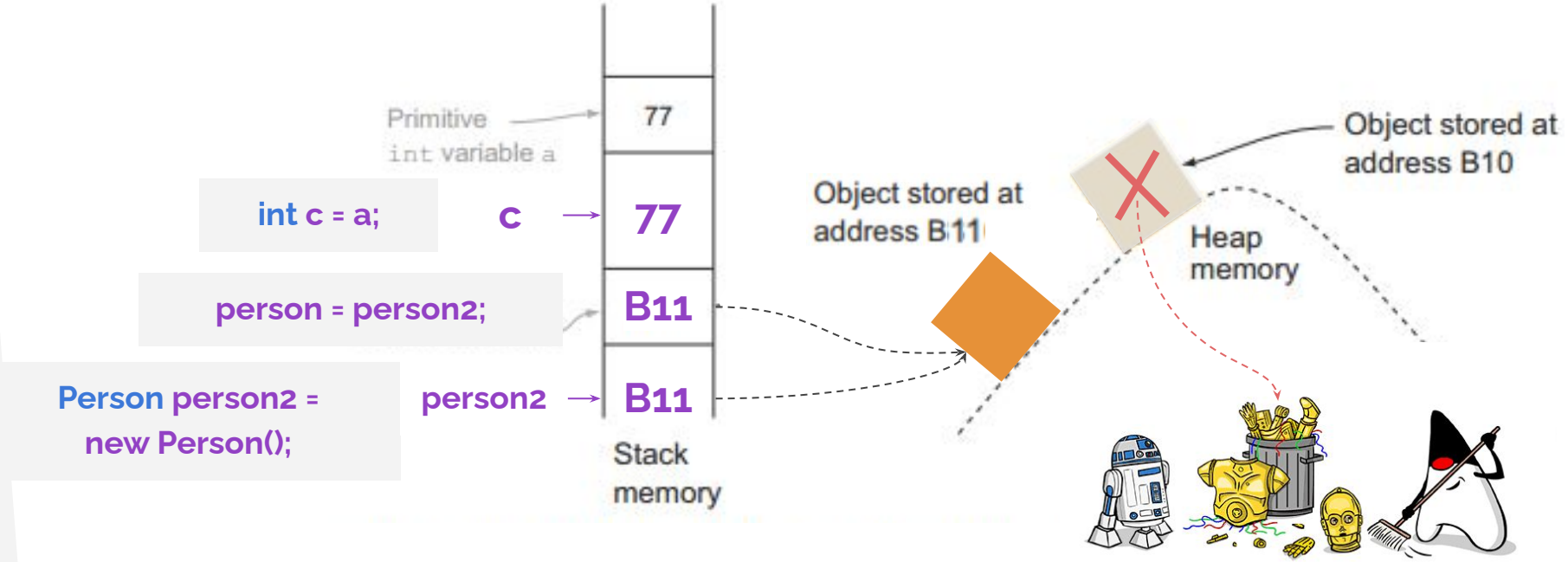
```java
public class Cachorro {
    String raca;
    float peso;
    float tamanho;
    float tamanhoDoPelo;
    Cor corDoPelo;

    Cachorro(String raca, float peso, float tamanho, float tamanhoDoPelo, Cor corDoPelo)
    {
        this.raca = raca;
        this.peso = peso;
        this.tamanho = tamanho;
        this.tamanhoDoPelo = tamanhoDoPelo;
        this.corDoPelo = corDoPelo;
    }
```

```java
Cor corLabrador = new Cor(170, 140, 110);
Cachorro labrador = new Cachorro("Labrador", 60.f, 0.5f, 0.03f, corLabrador);
```

Memória Stack e Heap



Lixo na Memória Heap e Garbage Collector



Conceito de Hoje

Conceito de Hoje

Tipos compostos de dados

Arrays,
ArrayLists,
Matrizes

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

por que usar tipos compostos?

String: Sequências de Caracteres

Nativamente a linguagem Java criou um tipo de dados novo para suportar sequências de caracteres. O nome deste tipo de dado é **String**.

Assim, em Java, **String é uma classe** e não um tipo primitivo/predefinido.

Operações com **Strings** ocorrem de acordo com o que essa classe possibilita.

String: Sintaxe

Classe

Variável **text** é um objeto, instância da classe String

Texto entre aspas que será atribuído ao objeto no momento seguinte à sua criação

```
String text = "Este é um exemplo de uma sequência de caracteres";
char caractere = text.charAt(3); // qual é esse caractere?
```

Tipo primitivo/ predefinido

Método da classe String que permite recuperar um char em uma posição específica

O valor da variável caractere é: 'e'

Como lidar com **muitos dados** do **mesmo tipo**?

Digamos que eu queira monitorar a nota de todos da turma.

Em determinado exercício que fizemos em sala, foi definido que **a nota de todos os 21 alunos da turma aumentaria em 0.5**.

Como faríamos isso em código?

21 DISCENTES FORAM MATRICULADOS NESSA TURMA

Matrícula	Nome	Curso	Situação
200721801	ALYSSON COSTA PEREIRA	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200721814	CESAR YURI E SILVA SANTOS	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200721804	ELIAS VITOR DA SILVA	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200721803	ENOCK DE MATOS LEAL NETO	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200721824	EVERSON EDUARDO DA SILVA ANDRADE	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200723884	FABIO LUIZ DA SILVA FILHO	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200721805	FELIPE BERTULINO DA SILVA	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200721797	FELIPE MUNIZ GARCEZ	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200721817	HELSON GONCALVES DOS SANTOS FILHO	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200721807	ITALO RYAN PIMENTEL HORAS	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200721816	JAMERSON CARLOS DOS SANTOS	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200723886	JOANA BEATRIZ SILVA BRANDAO	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200721820	JOAO BATISTA MENDES DA SILVA FILHO	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200721802	JOSE ALLEX DA SILVA MOURA	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO
200723865	JOSE MATEUS DE LIMA CHAGAS	ENGENHARIA DA COMPUTAÇÃO - UABJ - BELO JARDIM - BACHARELADO - Presencial - MT	MATRICULADO

Como lidar com **muitos dados** do **mesmo tipo**?

Digamos que eu queira monitorar a nota de todos da turma.

Em determinado exercício que fizemos em sala, foi definido que **a nota de todos os 21 alunos da turma aumentaria em 0.5**.

Como faríamos isso em código?

```
// cadastro das notas antes do exercício
float notaAluno01 = 9.0f;
float notaAluno02 = 9.5f;
float notaAluno03 = 8.5f;
//...
float notaAluno21 = 8.3f;


// acréscimo de meio ponto para todos
notaAluno01 += 0.5f;
notaAluno02 += 0.5f;
notaAluno03 += 0.5f;
//...
notaAluno21 += 0.5f;
```

Como lidar com **muitos dados** do **mesmo tipo**?

```
// cadastro das notas antes do exercício
float notaAluno01 = 9.0f;
float notaAluno02 = 9.5f;
float notaAluno03 = 8.5f;
//...
float notaAluno21 = 8.3f;


// acréscimo de meio ponto para todos
notaAluno01 += 0.5f;
notaAluno02 += 0.5f;
notaAluno03 += 0.5f;
//...
notaAluno21 += 0.5f;
```

E se fossem 50 alunos?

E se fosse um reitor de uma universidade com ~20.000 querendo calcular a média geral dos históricos escolares?

arrays

Arrays: Definição

Grupo de variáveis que contém valores todos **do mesmo tipo**

Os elementos de um array podem ser

- **tipos primitivos (ex.: int, float)** ou
- **tipos referência (ex.: classe Cachorro)**

Arrays: Definição

Grupo de variáveis que contém valores todos **do mesmo tipo**

```
// cadastro das notas antes do exercício
float notaAluno01 = 9.0f;
float notaAluno02 = 9.5f;
float notaAluno03 = 8.5f;
//...
float notaAluno21 = 8.3f;
```

sem array

```
// cadastro das notas antes do exercício
float[] notasAlunos = new float[21];
notasAlunos[0] = 9.0f;
notasAlunos[1] = 9.5f;
notasAlunos[2] = 8.5f;

notasAlunos[20] = 8.3f;
```

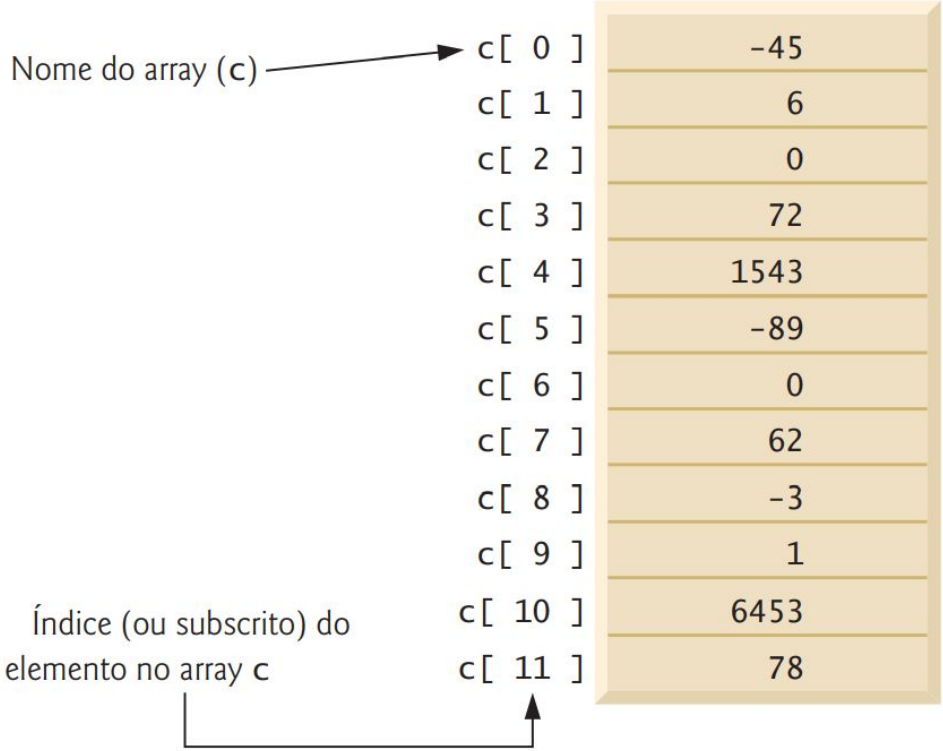
com array

Arrays: Acesso

Acessos aos elementos do array são feitos por **índices inteiros**
Com valores **entre *0* e *length - 1***
Expressões podem ser usadas como um índice

```
c[a + b] += 2;
```

Arrays: Acesso




```
c[a + b] += 2;
```

O que aconteceu na linha acima?

```
int a = 3;
int b = 4;
c[7] += 2;
c[7] == 64 //Essa comparação é true
```

Arrays: Acesso

Nome do array (c)

c[0]	-45
c[1]	6
c[2]	0
c[3]	72
c[4]	1543
c[5]	-89
c[6]	0
c[7]	62
c[8]	-3
c[9]	1
c[10]	6453
c[11]	78

Índice (ou subscrito) do elemento no array c

```
sum = c[0] + c[1] + c[2];
```

O que aconteceu na linha acima?

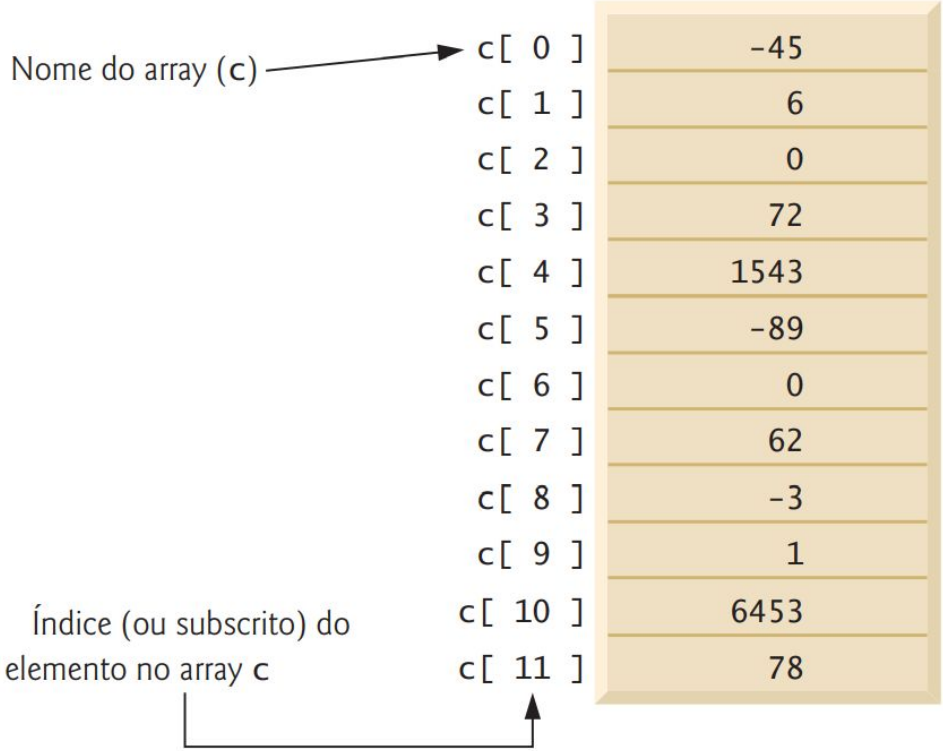
c[0] == -45

c[1] == 6

c[2] == 0

sum = -39

Arrays: Acesso

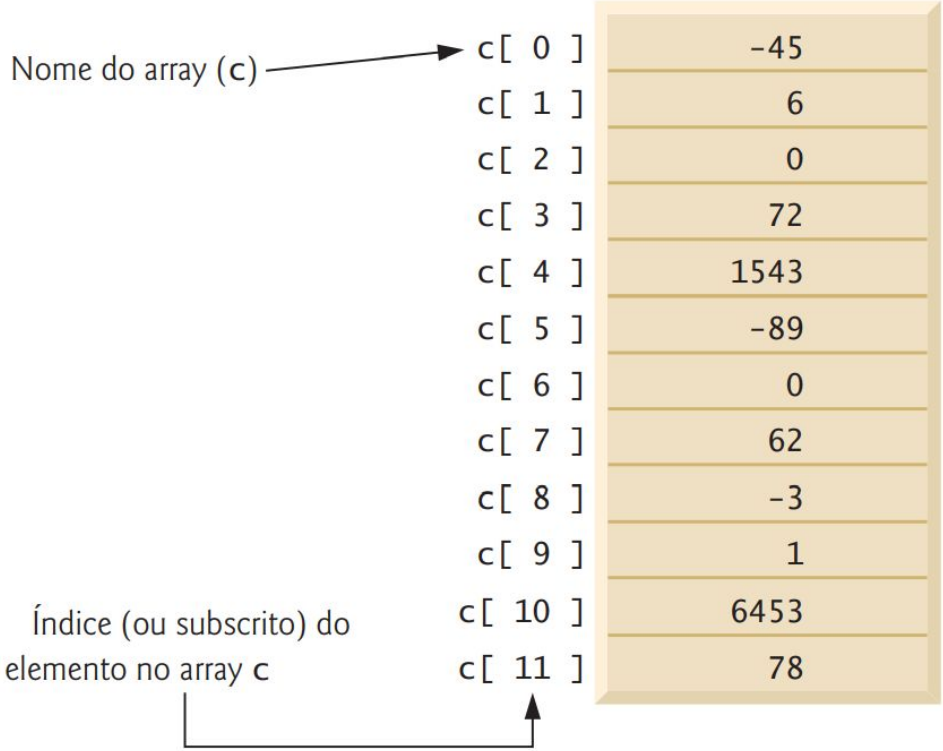



```
x = c[6] / 2;
```

O que aconteceu na linha acima?

x = 0

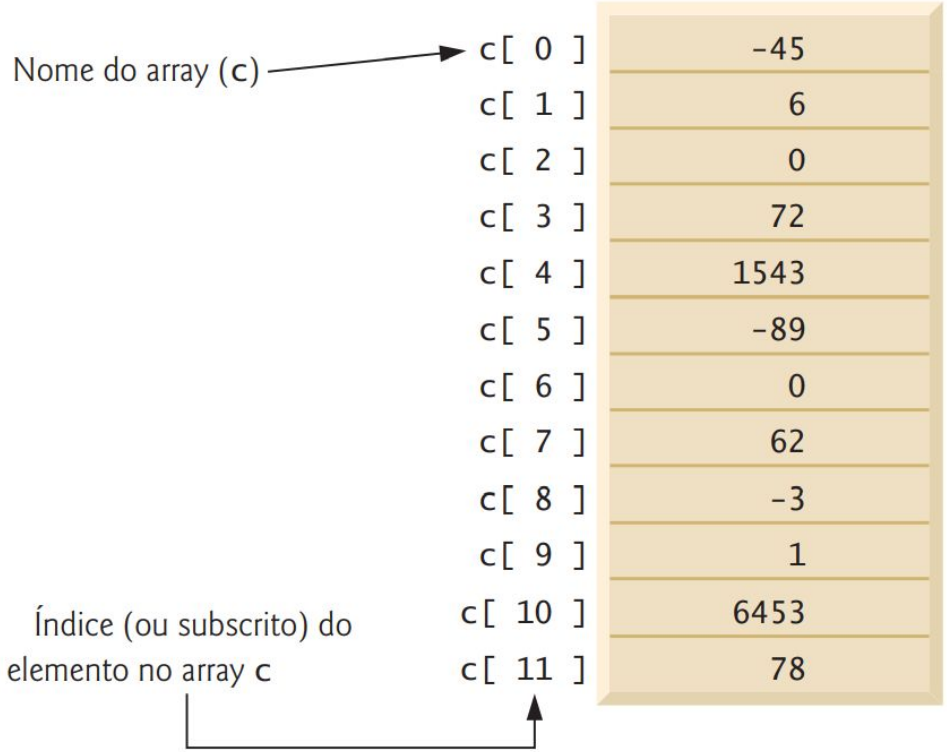
Arrays: Acesso




```
c[12] = 12;
```

O que aconteceu na linha acima?

Arrays: Acesso




```
c[12] = 12;
```

o que aconteceu na linha acima?

Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException:
Index 12 out of bounds for length 12
at Main.main(Main.java:12)

Arrays: Declaração: Sintaxe

Nome do objeto array

Com um número de elementos pré-definido

```
int[] arrayName = new int[21];
```

Tipo dos elementos dentro do array

Criação de um novo array

Arrays: Declaração

Declaração + criação e atribuição na mesma linha

```
int[] c = new int[12];
```

```
String[] b = new String[100]; // cria array b
String[] x = new String[27]; // cria array x
```

Declaração e depois a atribuição

```
int[] c; // declara a variável de array
c = new int[12]; // cria o array; atribui à variável de array
```

Arrays: Declaração e Lista de Inicializador

É possível já **atribuir valores específicos para os elementos** de um array no momento de sua declaração da seguinte forma

```
int[] n = { 10, 20, 30, 40, 50 };
```

Arrays: Iteração em Arrays com a Instrução **for**

No nosso caso inicial, ao usarmos arrays teríamos a mudança a seguir, com os **acessos** começando do **0**

```
// cadastro das notas antes do exercício
float notaAluno01 = 9.0f;
float notaAluno02 = 9.5f;
float notaAluno03 = 8.5f;
//...
float notaAluno21 = 8.3f;


// acréscimo de meio ponto para todos
notaAluno01 += 0.5f;
notaAluno02 += 0.5f;
notaAluno03 += 0.5f;
//...
notaAluno21 += 0.5f;
```

sem array

```
// cadastro das notas antes do exercício
float[] notasAlunos = new float[21];
float notasAlunos[0] = 9.0f;
float notasAlunos[1] = 9.5f;
float notasAlunos[2] = 8.5f;
//...
float notasAlunos[20] = 8.3f;


// acréscimo de meio ponto para todos
for(int i = 0; i < notasAlunos.length; i++)
{
    notasAlunos[i] += 0.5f;
}
```

com array

Arrays: **Iteração** em Arrays com a Instrução **for**

Existe um formato mais intuitivo para a instrução for quando se trata especificamente de **iterar** em arrays

```
// acréscimo de meio ponto para todos
for(int i = 0; i < notasAlunos.length; i++)
{
    notasAlunos[i] += 0.5f;
}
```

for com inteiro i para contagem

```
// acréscimo de um ponto para todos
for(float notaAluno : notasAlunos)
{
    notaAluno += 0.5f;
}
```

for que cria uma variável local interna ao for para cada elemento do array

Arrays: **Iteração** em Arrays com a Instrução **for**

Existe um formato mais intuitivo para a instrução for quando se trata especificamente de **iterar** em arrays

```
// acréscimo de meio ponto para todos
for(int i = 0; i < notasAlunos.length; i++)
{
    notasAlunos[i] += 0.5f;
}
```

for com inteiro i para contagem

```
float[] notasAlunos = new float[21];
float notasAlunos[0] = 9.0f;
float notasAlunos[1] = 9.5f;
float notasAlunos[2] = 8.5f;
for(float notaAluno : notasAlunos)
{
    notaAluno += 0.5f;
}
```

for que cria uma variável local interna ao f
para cada elemento do array

Problems Javadoc Declaration Console
<terminated> Main [Java Application] C:\Users\Lucas Fig
9.0
9.5
8.5

Apesar de mais elegante, a solução acima **não funciona** para o nosso propósito. notaAluno é uma variável local que existe só no escopo do for, é uma **cópia**

Tipos Primitivos e Referências na Memória

Tipos Primitivos em Java

- boolean
- byte
- short
- char
- int
- long
- float
- double

Tipos Referência em Java

- Todos os outros
  - Classes
    - String
  - Arrays
  - ...

```
// acréscimo de um ponto para todos
for(float notaAluno : notasAlunos)
{
    notaAluno += 0.5f;
}
```

for que cria uma variável local interna ao for para cada elemento do array

Apesar de mais elegante, a solução acima **não funciona** para o nosso propósito. notaAluno é uma variável local que existe só no escopo do for, é uma **cópia**

Tipos Primitivos e Referências na Memória

```
Person[] people = new Person[3];
people[0] = new Person(13);
people[1] = new Person(23);
people[2] = new Person(33);

for(var person : people)
{
    person.age += 1;
}

for(var person : people)
{
    System.out.println(person.age);
}
```

Problems Javadoc Declaration Console

```
<terminated> Main [Java Application] C:\Users\Lucas Figue
14
24
34
```

No caso ao lado podemos ver que por usarmos um **tipo referência (Classe Person)**, e acessarmos o conteúdo (**age**) do objeto que **estava na Heap**, o valor de fato **foi modificado** (incremento de 1) como mostrado no resultado do console.

Atenção! Se mudássemos a referência **person = new Person(14);** a alteração não persistiria. Por quê?

matrizes

Matrizes

É possível criarmos estruturas que são **Arrays multidimensionais** para representar dados específicos

Para isso construímos **Arrays de Arrays**
Esse conceito pode ser estendido para **N dimensões**
Arrays 2D são chamados de **Matrizes**

Matrizes

```
int[][] a = new int[3][4];
```

	Coluna 0	Coluna 1	Coluna 2	Coluna 3
Linha 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Linha 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Linha 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Índice de coluna
Índice de linha
Nome do array

Arrays 2D com Linhas de Diferentes Comprimentos

Também é possível ter tamanhos diferentes dos arrays em cada linha

Obs.: neste caso o array não se configura mais como uma matriz

```
int[][] b = {{1, 2}, {3, 4, 5}};
```

```
int[][] b = new int[2][];   // cria 2 linhas
b[0] = new int[5]; // cria 5 colunas para a linha 0
b[1] = new int[3]; // cria 3 colunas para a linha 1
```

arraylist

ArrayList

De forma similar a arrays, existem estruturas para armazenar coleções de dados chamadas de **listas** (ou vetores em alguns casos).

Em **termos teóricos** os listas visam cumprir a mesma função que os arrays

Em **termos práticos** estas estruturas trazem algumas facilidades

Segue um exemplo de declaração

```
ArrayList<String> list;
```

ArrayList: Métodos Disponíveis

Método	Descrição
`add`	Adiciona um elemento ao *final* do `ArrayList`.
`clear`	Remove todos os elementos do `ArrayList`.
`contains`	Retorna `true` se o `ArrayList` contém o elemento especificado; caso contrário, retorna `false`.
`get`	Retorna o elemento no índice especificado.
`indexOf`	Retorna o índice da primeira ocorrência do elemento especificado no `ArrayList`.
`remove`	Sobrecarregado. Remove a primeira ocorrência do valor especificado ou o elemento no índice especificado.
`Size`	Retorna o número de elementos armazenados em `ArrayList`.
`trimToSize`	Corta a capacidade do `ArrayList` para o número atual de elementos.

Figura 7.23 | Alguns métodos e propriedades da classe `ArrayList<T>`.

Exercício

Mini-Projeto 4

Criar um **array (ou ArrayList) de escolhas** como atributo da classe Capítulo

String[] escolhas;

O **construtor** da classe deve agora receber um array de escolhas

O método **mostrar()** deve mostrar todas as escolhas do array (ou nenhuma escolha no caso de um capítulo final).

Ao invés de retornar exclusivamente os valores 1 e 2, o método **escolher()** deve retornar o índice da escolha digitada (começando em 0)

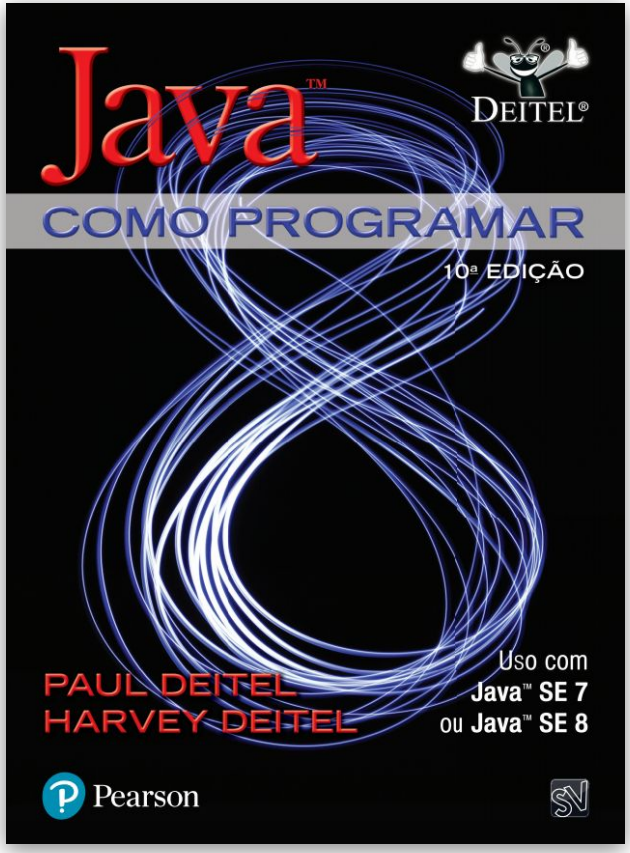
Referências

Java - Como Programar (10a Edição)

7 Arrays e ArrayLists

21.4 Listas encadeadas

21.7 Árvores

Ferramenta:

Eclipse: eclipse.org/downloads